

Retrospective Document

At the beginning of the project, I expected to build an AI agent that could reproduce bugs in a graphical application. The original idea was to take a GitHub issue, launch the application in a virtual machine, follow the reproduction steps, and save evidence for a human reviewer. That idea was interesting, but it also created a very large security and reliability problem. An agent that can control a computer, click through a graphical interface, and run commands has a broad attack surface. As the project developed, I changed the scope to a local security-review command-line tool. This was a better fit for the course because it still used AI in a practical engineering workflow, but it kept the system narrower, easier to test, and easier to secure.

Several parts of the project went well. The final system has a clear purpose: it helps a developer understand a local repository and triage GitHub issues or pull requests. The architecture also became much cleaner over time. Separating the tool into modules for scanning, configuration, GitHub access, AI calls, output, and error handling made the code easier to reason about. The threat model also helped guide implementation decisions. Instead of treating security as a final checklist, the project used the threat model to shape controls such as local credential storage, redaction, prompt boundaries, path confinement, dependency checks, and user-facing errors. The final result was a buildable and documented tool with tests, CI checks, and a clear issue log. This was a departure from my previous engineering efforts, which had been much more informally scoped and tested. I think I learnt a lot from formally following the software engineering process.

The main challenge was balancing useful AI behavior with safe engineering limits. It would have been easier to send more repository content directly to an AI provider, but that would increase privacy risk, cost, and failure rates. The final design uses smaller inputs, caps, redaction, batching, and intermediate summaries, which made the implementation more complicated. Another challenge was reliability with external AI providers. Long AI-assisted reports can run into rate limits, server errors, or inconsistent model behavior. The final release includes retries and fallback behavior, but this is still an area that needs improvement. I also learned that generated security reports are themselves sensitive artifacts because they can describe weak points in a system.

If I restarted the project, I would define the final scope earlier and avoid spending time on a design that was too broad for the course timeline. I would also build the adversarial prompt-injection tests earlier, because prompt injection is one of the most important risks for this kind of tool. I would add a stronger secret scanner earlier instead of relying mainly on regular expressions, and I would create live issue tracker entries throughout development instead of formalizing some of them near the end. Overall, the project taught me that AI tools are most useful when they are placed inside a narrow workflow with clear boundaries, human review, and strong operational controls. It also allowed me to apply the important software engineering skills that I had been learning over the course of the quarter in a practical setting.